

Code Ownership & IP: Otsuka's, From Day One

Client-owned IP is the default, not a negotiated outcome. Every line of code, every business rule, every configuration parameter is Otsuka's property from the moment it is written.

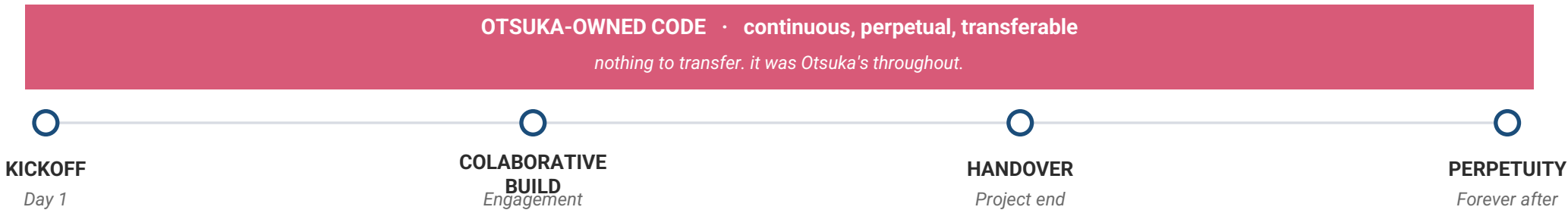
TYPICAL VENDOR MODEL

Code lives in the vendor's environment until handover.



DATAZYMES MODEL

Code is in Otsuka's environment from line one.



Inside Otsuka's repo from line one

Version-controlled in Otsuka's tooling. The full change history, review docs, and code are accessible to Otsuka throughout the engagement.



Perpetual, royalty-free, transferable rights

No usage cap, no license terms, no rebid clause. The same rights extend to all derived work, including future refresh cycles.



Open-source stack, no consultancy hooks

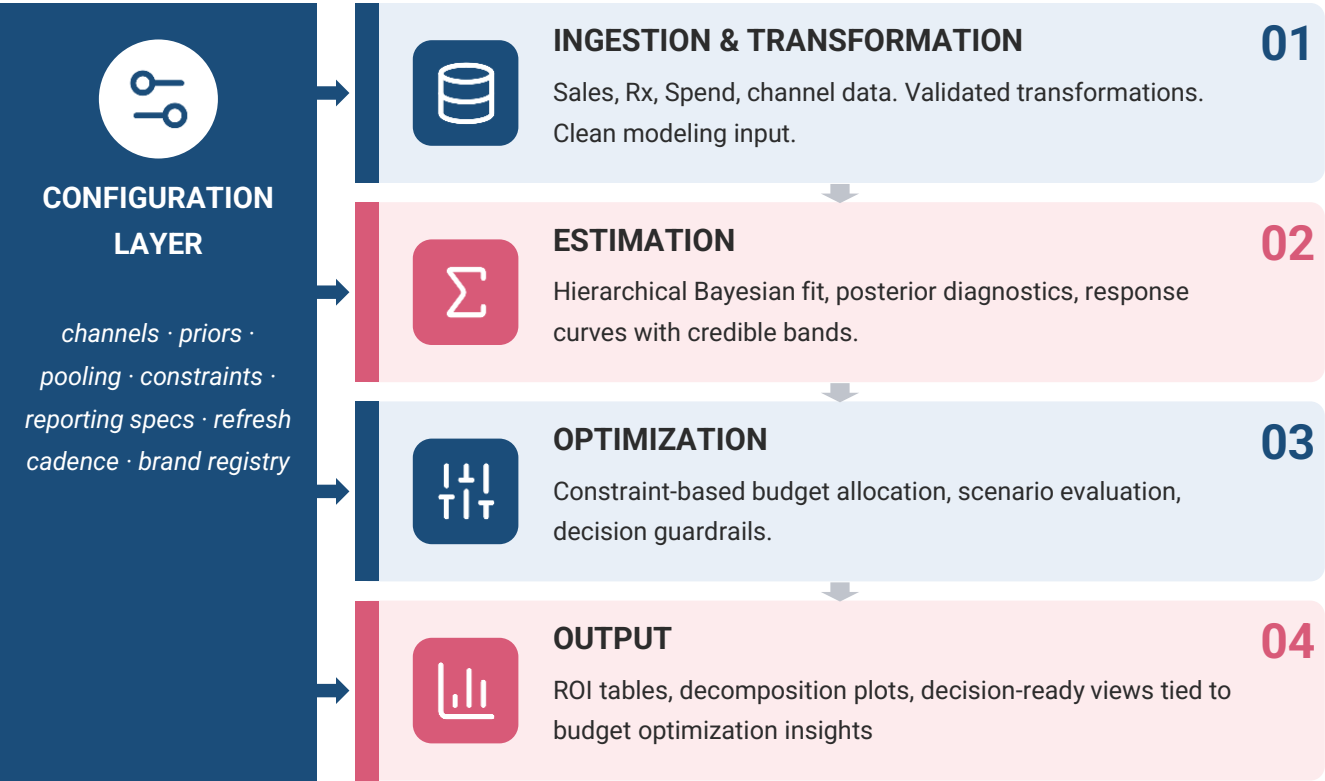
Standard libraries end to end. A version-locked dependency manifest sits alongside the code so any future environment can reproduce the build exactly.

ALREADY IN OPERATION On Metrics that Matter, the SQL checks, the CDE inventory, the validation logic, and the DIAs all sit inside Otsuka's environment, accessed and reviewed by Otsuka's team directly.

Modularity & Scalability: Configuration, Not Rebuild

The framework is built as four logical layers, each with a clear contract to the next. Channel definitions, model parameters, business rules, and reporting specs are all expressed as configuration. Adding, changing, or extending any of these is a parameter change.

FOUR LAYERS, ONE CONTRACT EACH



WHAT 'CONFIG, NOT CODE' MEANS IN PRACTICE

Each of these is absorbed at the configuration layer. The estimation, optimization, and output layers downstream stay untouched.

ADDING WITHOUT REBUILDING · config, not code	
• New channel	Register definition, mapping, and channel-specific transformations.
• New geography	Register grouping and pooling structure.
• New tactic	Channel sub-type with its own treatment.
• New brand	New configuration. Same codebase.
• Quarterly model evolution	Absorbed at the input contract. Downstream untouched.

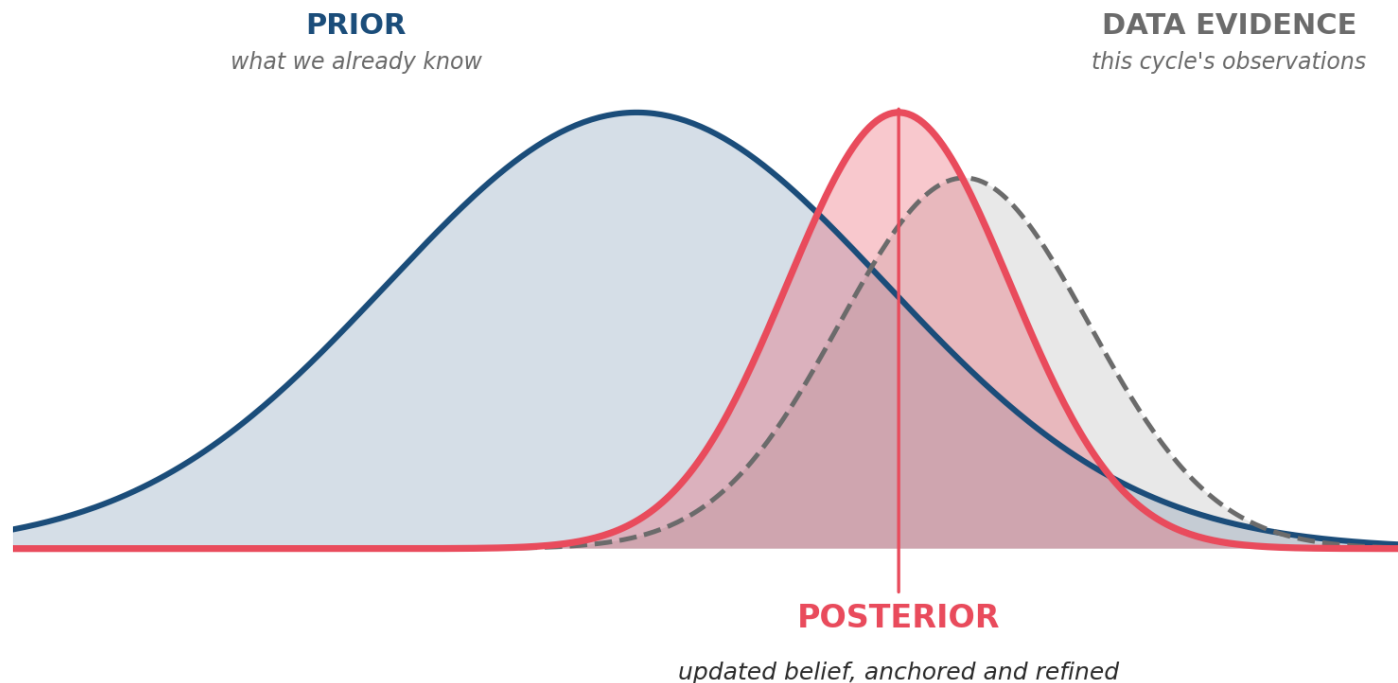
Same pattern for indications, portfolios, and refresh cadence changes.

EXPIRIENCE : Campaign Optimization Toolkit. Built for a top-five US pharma client, handed over with full documentation, and extended by the client's internal team across additional brands and planning cycles since, with no DataZymes involvement.

Why Bayesian fits a launch context

Limited promotional history, an evolving channel mix, and decisions to make in cycle. A Bayesian framework encodes what is known and refines it as new data arrives.

THE UPDATE MECHANIC



WHY IT FITS OTSUKA

01 Limited promotional history

Encode what prior cycles already know. Don't refit from short, noisy launch windows.

02 Evolving channel mix

Each refresh updates posteriors against the prior anchor. Institutional learning persists across cycles.

03 Decisions in cycle

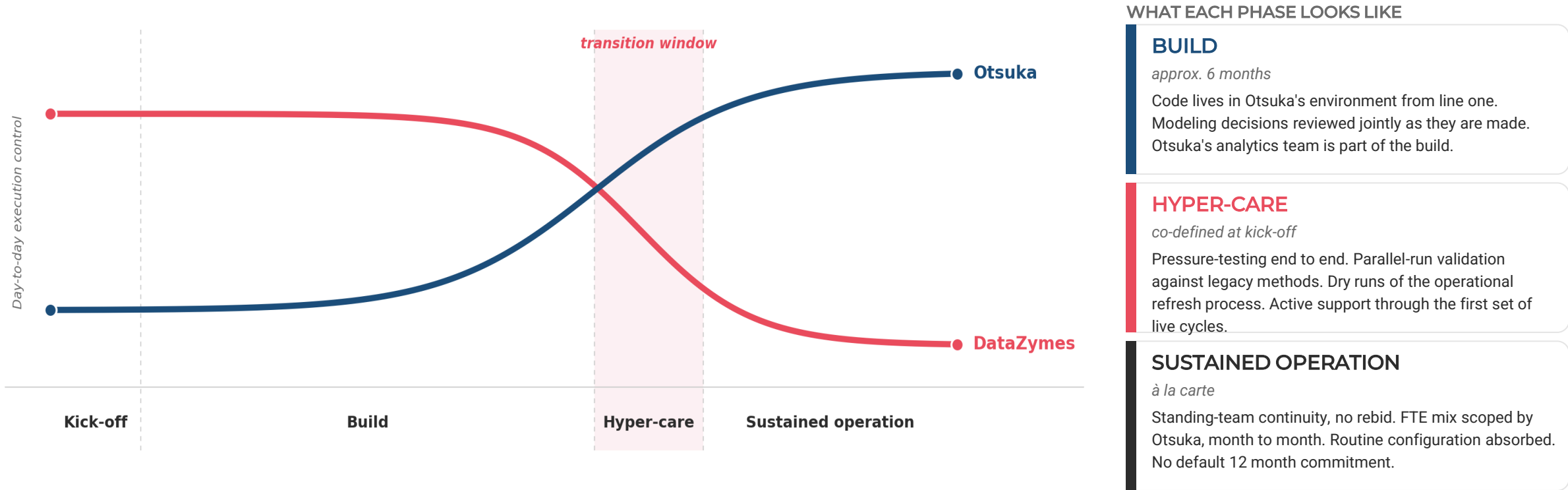
Posterior distributions quantify uncertainty directly. Credible intervals drive recommendations.

OTSUKA-OWNED FROM LINE ONE

Model spec, prior justifications, hierarchical config, and posterior diagnostics live in Otsuka's environment in perpetuity.

From build to autonomous operation

Hyper-care is structured as a transition, not a drop-off. By the time hyper-care begins, Otsuka's team has been close enough to the build that running the cycle independently is a continuation of how they have already been working.

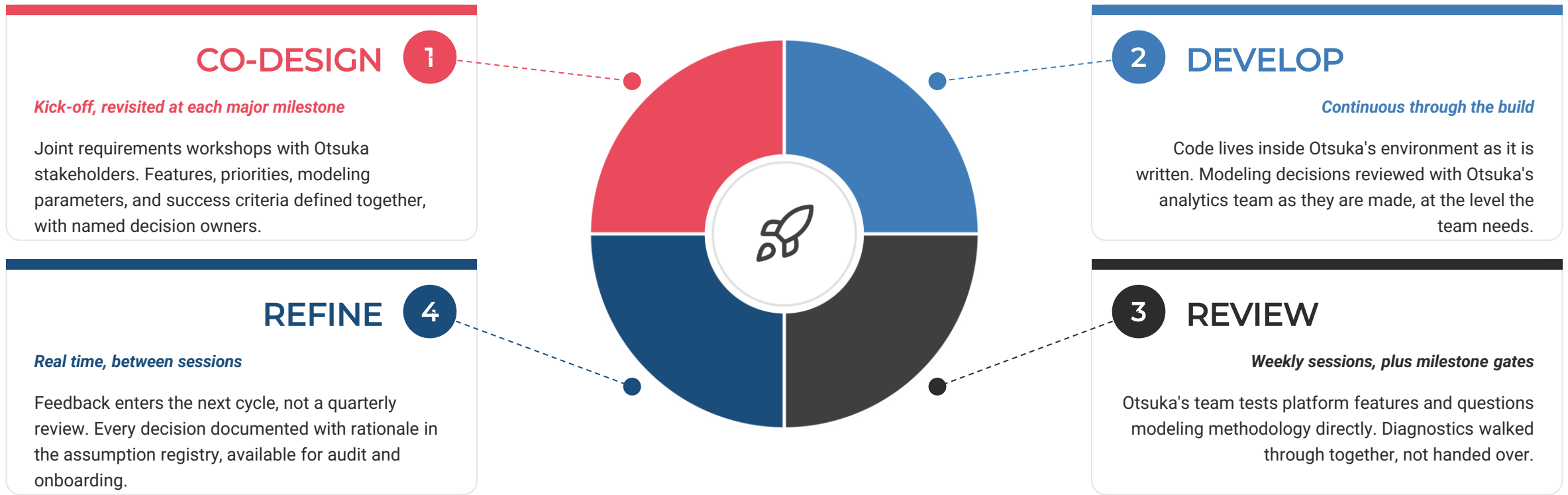


There is no single handoff event because there is nothing waiting to be transferred. Capability builds inside Otsuka's team across every cycle of the build. Hyper-care confirms the team can run the cycle independently. Sustained operation runs at whatever cadence Otsuka chooses to engage at, with the same standing team that built the work.

TL;DR Build, hyper-care, sustained operation. The shape and length of hyper-care are scaled to the complexity of the build and the readiness of Otsuka's team, not to a fixed handoff event.

Joint development process*

An iterative working pattern that runs across every build cycle. Feedback travels in real time, not at quarterly checkpoints. Every decision documented with rationale for future reference.



*Applies whether the build runs through a Dataiku code-based automation route or a custom build path. The working pattern above governs the cycle in either case.

TL;DR Co-design, develop, review, refine. Real-time feedback incorporation, not quarterly review cycles. Every decision documented with rationale for future reference and training.